

Project Status & Next-Phase Roadmap

AI-agent readiness scanner — where we are, what's missing, and what to build next.

Prepared	20 July 2026
Scope	Full-stack status audit + prioritized roadmap
Repository	UTSAV-2024/visum (main @ PR #3 merged)
Status	Scanner live & honest · dashboard still sample data
Audience	Internal — product & engineering

89/100

Dogfood score (local prod)

421

Backend tests passing

8

AI-readiness checks

OVERVIEW

Executive summary

Visum scans any public website for **AI-agent readiness** — how well AI systems (ChatGPT, Claude, Perplexity) can find, read, and cite it — and returns a 0–100 score with per-check findings and fixes in about 20 seconds. The scanner engine is complete, accurate, and now **honest**: it never fabricates a score it couldn't measure. Visum's own site scores **89/100** against its own scanner. A full email/password **authentication** system was just added.

The central gap is that the **product dashboard is still sample data**. A user can now sign up and log in, but cannot yet see their *own* scans. Closing that gap — wiring the dashboard to real per-user data — is the single highest-leverage next phase, and is detailed in Section 5.

SOLID	Scanner backend — 8 checks, honest scoring, SSRF-guarded, 421 tests, in CI.
SOLID	Scan experience — Landing → scan → result report with trustworthy copy & numbers.
SOLID	Own-site readiness — 89/100 dogfood; real MCP endpoint, JSON-LD, llms.txt, OpenAPI.
NEW	Authentication — Supabase signup/login, guards, logout — opt-in via env vars.
MOCK	Product dashboard — Analytics/insights/competitors etc. are sample data behind banners.
MISSING	Per-user data — Auth exists but the dashboard doesn't show a user's own scans yet.

CONTEXT

1 · What Visum is

A hosted scanner plus a product surface around it. Architecture:

Layer	Stack & hosting
Frontend	Next.js 16 (Pages Router), React 19, Tailwind v4 — hosted on Vercel (visum-eight.vercel.app).
Backend	FastAPI + Playwright (Chromium) + httpx — hosted on Render (visum-xoe3.onrender.com).
Data	Supabase (Postgres) — scan persistence and, now, authentication.
Analytics	PostHog + Vercel Analytics.

The eight checks

robots.txt (AI-bot permissions) · JSON-LD structured data · llms.txt · MCP endpoint · JavaScript rendering · meta / Open Graph tags · sitemap.xml · page load speed. Each returns points, a plain-English finding, and a concrete fix; the total is normalized to 100.

Core funnel

Landing → enter a URL → scan → email gate → full result report. This funnel is public and requires no account.

DONE

2 · What we have

Scanner & scoring

- All 8 checks implemented with **honest measurement**: when tooling can't measure a check (e.g. the headless browser is unavailable), it reports *not measured* and is excluded from the total instead of faking a score.
- Correct, defensible scoring: missing robots.txt = "all crawlers allowed" (passes); MCP is a right-weighted bonus signal; soft-404 detection stops SPA shells counting as real files.
- Security: URL validation blocks localhost/private/loopback ranges (SSRF); per-IP rate limiting and a concurrency semaphore.
- Quality gate: 421 backend tests run on every push/PR via GitHub Actions.

Scan experience & own-site readiness

- A trustworthy result report: unique per-check "why it matters", severity-consistent impact text, score gains capped to points available, an average that matches the headline, and a neutral "not measured" state.
- Visum dogfoods to **89/100** — every content-controllable check maxed. It ships a real MCP endpoint at /api/mcp (JSON-RPC), an OpenAPI spec, rich JSON-LD, and a complete llms.txt.
- Security headers (CSP, X-Frame-Options, etc.) across all routes.

Authentication (new, opt-in)

- Supabase Auth via @supabase/ssr: /login and /signup (with email-confirmation flow), a useAuth() context, a client-side route guard, and an optimistic proxy.js cookie guard.
- Working logout and real user display in the sidebar.
- Opt-in: inactive until the two NEXT_PUBLIC_SUPABASE_* env vars are set; until then the app runs exactly as before. /result stays public.

GAPS

3 · What we lack

A · Product is a demo shell where it counts

- The dashboard is entirely mock data (analytics, insights, recommendations, competitors, reports, team, etc.).
- Auth ≠ personalized data yet — no "my scans" view backed by the Supabase scans table.
- Billing, API keys, Integrations, and Settings are "Soon" stubs.

B · Data & persistence

- Backend scan status and rate-limiter state are in-memory — lost on restart, not shared across instances.
- No documented Row Level Security (RLS) policies on the scans table.

C · Auth completeness & hardening

- No password-reset / forgot-password flow; no social (OAuth) login.
- Route protection is client-side + optimistic proxy only — no server-side enforcement (fine for sample data, not for real data).
- Supabase service-role key rotation still pending (owner action).

D · Infra & quality

- CI tests the backend only — no frontend build/lint/typecheck or test runner in CI.
- No error monitoring (e.g. Sentry) on either tier; no end-to-end tests.
- The next dev server has an HMR crash loop; all verification currently goes through production builds.

E · Go-to-market

- Deployed 90+ dogfood score not yet confirmed on production (a strong proof point once verified).
- Stripe payment link is optional/unconfigured — no real Pro upgrade path.

PLAN

4 · Roadmap

Prioritized by leverage. P0 turns Visum from a convincing demo into a real product.

P0**Make it a real product**

#	Item	Why it matters
1	Wire the dashboard to real per-user scan data	The biggest gap. Show the logged-in user's own scans, score history, and per-scan detail.
2	Server-side auth + Row Level Security	getServerSideProps protection + RLS so a user can only read their own rows.
3	Deploy & verify the 90+ dogfood score	Confirm on production and use it as a landing-page proof point.
4	Rotate the Supabase service-role key	Owner action; closes the last outstanding security item.

P1**Trust, retention, resilience**

#	Item	Why it matters
1	Password reset + one OAuth provider	Table-stakes auth completeness and lower signup friction.
2	Persist backend state (Supabase/Redis)	Scan status and rate-limits survive restarts and scale across instances.
3	Frontend in CI + a smoke E2E	Catch build/lint/type regressions and protect the scan & login flows.
4	Fix the dev-server HMR crash	Restore live local preview instead of relying on production builds.

P2**Polish & scale**

#	Item	Why it matters
1	Build out or remove Billing/API/Integrations/Settings	Don't ship permanent "Soon" tags.
2	Error monitoring (Sentry) on both tiers	See failures in the wild, not just analytics.
3	Harden the MCP endpoint for production load	Slow scans can exceed serverless timeouts; go async + status polling.

NEXT PHASE

5 · Deep dive — wire the dashboard to real data

This is the recommended next phase. Authentication is in place, so the work is to make a signed-in user's experience reflect **their own** scans instead of sample data — and to make that data access safe on the server. Target outcome: a user signs up, runs a scan, and sees it in their dashboard with history over time.

5.1 · Data model

A scans table already exists (the email gate upserts into it). Extend it to associate scans with an authenticated user and to store the full result for history.

Column	Definition & notes
id	uuid, primary key.
user_id	uuid, FK → auth.users. Nullable for anonymous funnel scans; set when a logged-in user scans.
url	text — the scanned URL.
total_score	int — 0–100.
band	text — score band label.
checks	jsonb — the full per-check result payload.
created_at	timestampz, default now().

5.2 · Row Level Security (the safety-critical piece)

- Enable RLS on scans. Policy: a user may **select** rows where `user_id = auth.uid()`.
- Writes from the logged-in browser go through the anon key + RLS (insert with `user_id = auth.uid()`).
- Keep the existing service-key server route for the anonymous funnel, but never expose the service key to the browser.

5.3 · Server-side enforcement

- Use the included `lib/supabase/server.js` client in `getServerSideProps` for `/dashboard` and any real-data route, so protection no longer relies only on the client guard.
- Redirect unauthenticated requests to `/login` server-side; fetch the user's scans there and pass as props.

5.4 · UI — a "My Scans" view

- Replace the mock dashboard landing with the user's real scans: a list (URL, score, band, date), a score-over-time sparkline, and a click-through to the existing result report.
- Empty state: "Run your first scan" CTA that drops the user into the scan flow.
- On completing a scan while logged in, persist it against `user_id` and route to the report.

5.5 · Acceptance criteria

- A new user can sign up, run a scan, and see exactly that scan in their dashboard — and nobody else's.
- Direct-navigating to `/dashboard` while logged out redirects to `/login` on the server (no protected shell flashes).
- RLS verified: querying another user's row returns nothing even with a valid session.
- Existing public funnel (scan → email → result) still works unchanged.

5.6 · Suggested sequence

Step	Work
------	------

Step 1	Migration: extend scans + enable RLS + policies (Supabase).
Step 2	Persist logged-in scans with user_id (browser client + RLS insert).
Step 3	getServerSideProps auth + fetch user scans for /dashboard.
Step 4	Build the "My Scans" list + empty state + detail link.
Step 5	Rotate the service-role key; add a frontend CI check.

REFERENCE

Appendix · What shipped this cycle (PR #3)

Area	Summary
Honest scoring & copy	measured=False for unmeasurable checks; weighted totals; robots/MCP fixes; trustworthy result copy.
Dogfood to 89/100	Real /api/mcp endpoint, /openapi.json, richer JSON-LD, fixed llms.txt, nav audit.
MCP hardening	Reject non-http(s) and internal hosts before calling the backend.
Authentication	Supabase signup/login, route guard, proxy guard, working logout — opt-in.
Docs	PROJECT-STATUS.md and IMPROVEMENT-CHECKLIST.md track ongoing work.

Key paths

Auth: `lib/auth-context.js`, `lib/supabase/{client, server}.js`, `pages/{login, signup}.js`, `components/auth/route-guard.tsx`, `proxy.js`. Scanner: `backend/app/checks/*`, `backend/app/scorer.py`, `backend/app/crawler.py`. MCP endpoint: `frontend/pages/api/mcp.js`.

Set `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` (Supabase → Settings → API) to activate authentication. Enable Email auth in the Supabase dashboard.